M2-2: High Performance Computing Project Report

Sorin Bețișor, Ethan Goetsch, David Wicker, Botond Moksony, Mikle Kuzin, Antoni Sienkiewicz, and Lakshana Sivaprakash

Maastricht University, Department of Advanced Computer Sciences

Contents

Abstract

Computational Fluid Dynamics (CFD) underpins applications ranging from aerodynamic shape optimisation to marine-craft maneuvering and on-board flow control, all of which demand solutions that are both accurate and fast. At the core of our workflow lies a **configurable, second-order finite-difference solver** for the incompressible Navier–Stokes equations in vorticity–stream-function form. Written in `C`, the solver accepts problem geometry, Reynolds number, wall motions, body forces, and obstacle layouts through a lightweight configuration, enabling a wide spectrum of two-dimensional test cases, not just the lid-driven cavity, without code changes. High-performance-computing features are integrated from the ground up: OpenMP exposes thread-level parallelism on multicore CPUs, while Vulkan compute shaders and CUDA kernels off-load the most expensive kernels to consumer-grade GPUs, lowering iteration time to 9 ms.

To remove the remaining Poisson bottleneck, a **Physics-Informed Neural Network** (PINN) is trained on snapshots produced by the solver and then substituted for the stream-function solve in a follow-up run. This hybrid arrangement preserves the solver's verifiable fidelity while more than halving the total iteration cost. By combining a flexible, hardware-optimised CFD engine with a post-trained neural surrogate, the workflow delivers high-quality vorticity, velocity and stream-function fields at interactive rates, paving the way for real-time optimisation and control in industrial CFD practice.

1 Introduction

The accurate numerical simulation of fluid flow underpins progress in engineering, meteorology and the physical sciences. Classical discretisation techniques, finite-difference, finite-volume and finite-element schemes, remain the gold standard for robustness and fidelity, but their computational cost grows rapidly with mesh resolution and geometric complexity [1, 2]. Even on modern hardware, achieving the spatio-temporal detail demanded by real-time design or decision-making workflows can be prohibitive.

High-Performance Computing (HPC) ameliorates some of these limitations by exploiting thread-level parallelism, vectorisation and GPU off-loading. Recent large-scale studies demonstrate order-of-magnitude speed-ups for classical solvers while preserving accuracy, thereby establishing a strong baseline against which alternative methods must be measured [3]. We build on this tradition: a C finite-volume solver has been implemented, providing both high-fidelity data and a benchmark for subsequent comparisons.

Over the last decade, machine-learning approaches, most notably Physics-Informed Neural Networks (PINNs), have emerged as a compelling complement to traditional solvers. By embedding the governing partial differential equations in the loss function, PINNs learn solution manifolds without requiring dens labeled data [4]. Extensions have tackled high-speed and turbulent regimes [5, 6], mitigated gradient-flow pathologies [7], and coupled differentiable simulators with graph neural networks for improved generalisation [8]. Comprehensive surveys chart both successes and open challenges, particularly regarding training cost and geometrical complexity [9].

Research questions Guided by the foregoing motivation, our study addresses the following questions:

- *Solver performance.* Relative to established state-of-the-art reference codes [10], what wall-clock speed-up and numerical fidelity can an OpenMP-/Vulkan-accelerated C implementation deliver on the lid-driven cavity benchmark [section 2.1]?
- *PINN-Accelerated CFD Performance and Accuracy.* To what extent can a Physics-Informed Neural Network (PINN), when integrated into a CFD simulation pipeline as a surrogate for computationally intensive sub-domains, reduce overall runtime and enhance performance metrics while maintaining solution accuracy within predefined tolerances?

2 Methods

Our workflow blends classical CFD with modern machine learning through two interlocking components:

1. **CFD C solver for Vorticity, Velocity and Stream Function**, a second-order finite-difference code for incompressible flow, written in C and accelerated on CPUs (OpenMP) and GPUs (Vulkan). It produces the high-fidelity snapshots used for verification and for training neural surrogates.

2. **Physics-Informed Neural Network (PINN)** , a deep network whose loss function embeds the Navier–Stokes equations and boundary conditions, allowing it to infer flow fields from sparse data and generalise across parameter regimes.

2.1 Benchmark problem: Lid-Driven Cavity

The benchmark we are using centres on the classical *lid-driven cavity* test. A square cavity of side length L is sealed on all sides; the top wall (the "lid") translates horizontally at a constant speed U_0 while the remaining walls remain stationary and enforce no-slip conditions. Although no analytic solution exists, the flow is well documented in the literature, making it ideal for code verification and cross-comparison [1].

2.2 Governing equations

The motion of an incompressible, Newtonian fluid in two spatial dimensions is described by the Navier–Stokes equations. For numerical efficiency, we recast them in *vorticity–streamfunction* form, which removes pressure from the unknown set and enforces mass conservation by construction:

$$\frac{\partial \omega}{\partial t} + \mathbf{u} \cdot \nabla \omega = \frac{1}{\mathrm{Re}} \nabla^2 \omega \tag{1}$$

$$\nabla^2 \psi = -\omega \tag{2}$$

$$\mathbf{u} = (\partial_y \psi,\ -\partial_x \psi) \tag{3}$$

Here ω is the scalar vorticity, ψ the streamfunction, $\mathbf{u} = (u, v)$ the velocity field, and ∇^2 the two-dimensional Laplacian. The system is discretised on a uniform Cartesian grid with second-order finite differences;

Reynolds number.

$$\mathrm{Re} \ = \ \frac{U_0 L}{\nu},$$

with U_0 a characteristic velocity, L a reference length and ν the kinematic viscosity. It expresses the balance between inertial and viscous forces and therefore dictates the qualitative flow regime [2].

2.3 Boundary conditions (overview)

Solid walls impose the no-slip condition $\mathbf{u} = \mathbf{0}$ and impermeability $\mathbf{u} \cdot \mathbf{n} = 0$. Any moving boundary simply prescribes its tangential velocity in $\mathbf{u}$.

Vorticity at walls is reconstructed from wall-normal derivatives of ψ to retain second-order accuracy, while the Poisson problem (2) is solved each time step using Jacobi or Gauss–Seidel iterations. These treatments ensure that mass conservation and momentum diffusion are respected at the discrete level.

2.4 Solver workflow

At every time step, the vorticity field is first advanced with an explicit finite-difference update. This leaves a discrete Poisson problem for the stream-function, which is solved by the Gauss–Seidel numerical solver (algorithm 2); the iteration sweeps cell-by-cell until the residual satisfies a specified tolerance. Differentiating the resulting stream-function produces the velocity field, which is then written to VTK[1] files and as input to the Neural Network. The complete workflow is presented in algorithm 1.

Algorithm 1 Solver workflow time step (Velocity + Vorticity)

Input: ω^n, ψ^n, $\mathbf{u}^n$, Δx, Δy, Δt
Output: ω^{n+1}, ψ^{n+1}, $\mathbf{u}^{n+1}$

1: Apply lid speed $u = U_0$, $v = 0$.
2: Update interior vorticity with explicit Euler.
3: Solve $\nabla^2\psi = -\omega$ using Gauss-Seidel.
4: Recover velocity: $u = \partial_y\psi$, $v = -\partial_x\psi$.
5: Write velocity to VTK.

Algorithm 2 Gauss–Seidel for the Poisson equation (Stream Function)

Input: Initial guess $\psi^{(0)}$, $-\omega$, tolerance `tol`, maximum sweeps `max_it`, relaxation factor ω_{SOR}
Output: Approximate solution $\psi^{(k)}$

1: $k \leftarrow 0$, $res \leftarrow 10\,$`tol`
2: **while** $res >$ `tol` **and** $k <$ `max_it` **do**
3: For all interior (i, j):

$$\psi^*_{i,j} = \frac{\Delta y^2(\psi_{i+1,j} + \psi_{i-1,j}) + \Delta x^2(\psi_{i,j+1} + \psi_{i,j-1}) + \Delta x^2\Delta y^2\,\omega_{i,j}}{2(\Delta x^2 + \Delta y^2)};$$

4: $\psi_{i,j} \leftarrow (1 - \omega_{\text{SOR}})\psi_{i,j} + \omega_{\text{SOR}}\,\psi^*_{i,j}$ ▷ in-place update
5: After the sweep, update wall vorticity.
6: $res \leftarrow \|\nabla^2\psi + \omega\|_2 / \|\omega\|_2$
7: $k \leftarrow k + 1$
8: **end while**

2.5 CPU Parallelization - OpenMP (Solver)

OpenMP support is enabled at compile time (`-fopenmp`) and can be toggled at run time.

- **Loop annotation.** All compute-intensive loops (vorticity advection, Poisson SOR sweeps, Jacobi smoothing and vector norms) are wrapped in `#pragma omp parallel for` directives, typically with `collapse(2)` to expose the 2-D grid
- **Red–black SOR.** The Poisson solver uses a red–black colouring so that adjacent nodes are never updated simultaneously, removing race conditions without explicit locks.
- **Size threshold.** For very small grids ($n < 128$) the overhead outweighs the benefit; therefore each kernel checks the problem size and falls back to the serial path if the domain is too small.

With *16* CPU threads the OpenMP variant achieves a 2× speed-up on the 256^2 benchmark while retaining identical numerical results.

[1]VTK (Visualization Toolkit) files hold structured or unstructured grid geometry plus per-node or per-cell scalar/vector data, and they are read natively by ParaView for interactive flow-field visualisation.

2.6 GPU Offloading - Vulkan & CUDA (Solver)

To accelerate the most expensive kernels of the cavity solver we ported Algorithm 1 to a **Vulkan-compute** back-end. Vulkan was selected for its low-level, cross-platform access to GPU resources and its tight control over synchronisation and memory layout, which translate into predictable performance for general-purpose compute workloads. The entire update loop, vorticity advection, Poisson iteration, and velocity reconstruction, is executed inside multiple compute shaders, thereby avoiding repeated CPU-GPU transfers.

A dedicated `VulkanEngine` class initialises device queues, descriptor sets, and storage buffers for the primary fields: velocity, vorticity, stream function, and a scratch buffer. During each sub-step, threads write intermediates to the scratch buffer; after an intra-work-group barrier, those values are copied back into the field's main buffer, ensuring race-free in-place updates. Because the simulation buffers reside permanently in device memory, a staging buffer is required only when data must be retrieved for I/O. Every ten time steps the engine pauses the kernel, copies the current fields through the staging buffer, and saves them to `.vtk` files for later visualisation.

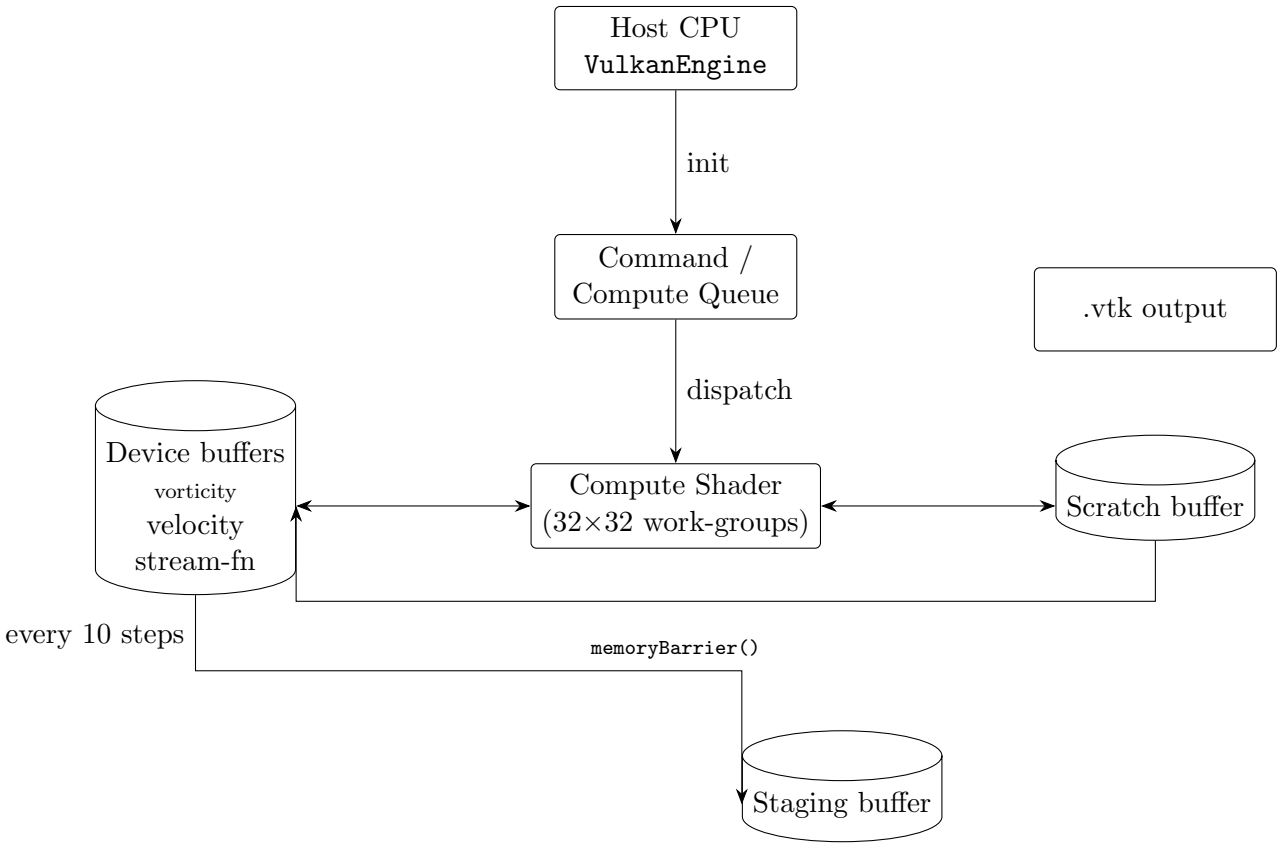


Figure 1: GPU-offloaded pipeline for one-time step of the cavity solver.

For comparison we also implemented a straightforward **CUDA** port: each numerical stage, vorticity advection, red–black SOR sweeps for the Poisson equation, and velocity reconstruction, is launched as a separate kernel from the host. All field arrays reside in pinned device memory, so the only host-device transfers are a periodic snapshot every ten iterations. Kernel grids use one thread per cell with 16×16 blocks, while a single CUDA stream ensures in-order execution without extra synchronisation.

2.7 Neural Network (PINN)

To accelerate the most expensive portions of the CFD update while retaining physical fidelity, we employ a *Physics-Informed Neural Network* (PINN) [4]. Unlike conventional data-driven models, a PINN incorporates the governing Navier–Stokes equations directly into its loss function, enabling accurate flow predictions from relatively small training sets.

Network architecture. We adopt a fully connected feed-forward topology with L hidden layers, each containing n_h neurons and `tanh` activations, a configuration shown to balance universal approximation capability with stable training for PDE problems [5]. The input vector comprises spatial coordinates (x, y) and time t, while the network outputs approximate values of the stream-function ψ_θ and vorticity ω_θ, where θ denotes all trainable weights and biases.

Physics-aware loss. The training objective combines three terms:

$$\mathcal{L}(\theta) = \lambda_d \mathcal{L}_{\text{data}} + \lambda_p \mathcal{L}_{\text{PDE}} + \lambda_b \mathcal{L}_{\text{BC}},$$

where

- $\mathcal{L}_{\text{data}}$ penalises the ℓ_2 error between network outputs and reference snapshots generated by the classical solver;
- $\mathcal{L}_{\text{PDE}}$ enforces the vorticity–transport and Poisson residuals using automatic differentiation;
- $\mathcal{L}_{\text{BC}}$ imposes no-slip and lid-velocity conditions along the cavity walls.

The coefficients $\lambda_d, \lambda_p, \lambda_b$ are tuned to balance data fidelity and physics consistency.

Expected benefits. By embedding physical constraints, the PINN generalises beyond its training set and provides smooth, divergence-free fields, making it a strong candidate to replace the linear Poisson solve or other stiff components of the classical algorithm [8]. Comprehensive reviews highlight both the promise and the current limits of PINN technology in fluid dynamics applications [9]; our implementation follows their best-practice guidelines for network depth, initialisation, and residual weighting.

3 Experiments

3.1 Solver Verification: Numerical Accuracy Benchmark

To verify that every back-end produces an identical flow solution, we compare key benchmark quantities against high-resolution reference data of Ghia *et al.* [10]. For each run we record

- the co-ordinates of the primary vortex centre (x_c, y_c),
- the peak non-dimensional stream-function $\psi_{\max}$, and
- the domain-wide ℓ_2 error in vorticity, $\|\omega - \omega_{\text{ref}}\|_2$,

all sampled at the final time step. If every configuration stays within $< 1\,\%$ of the reference for (x_c, y_c) and $\psi_{\max}$, and the vorticity norm differs by less than 5×10^{-4}, this will confirm that the solver's the second-order scheme retains its design accuracy regardless of the execution target.

3.2 Performance Analysis: two grid sizes

We benchmark the solver on two meshes: a small grid of 64×64 cells ($\approx 4\,000$ control volumes) and a medium grid of 256×256 cells ($\approx 65\,000$ volumes). Each case is executed on the same test machine, **Ryzen 7 6000 CPU** and **RTX 3050 Laptop GPU** , for 351 iterations. Four back-ends are timed: (i) **Serial C on a single core**, (ii) **OpenMP using 16 CPU threads**, (iii) **Vulkan compute shaders on the GPU**, and (iv) **CUDA on the same GPU**. **The same configuration and test** (classical benchmark - lid-driven cavity) was used for all instances. After the run, we parse the text logs and compute the mean and sample standard deviation of the "Iter time" field for every back end. For example, a typical log entry reads:

```
Poisson equation solved with 106 iterations { root-sum-of-squares error:  9.538705E-04
Iteration:  21 | Time:  0.105000 | Progress:  0.53% | Iter time:  0.499 s
Continuity max:  3.538836E-15 | Continuity min:  -3.073930E-15
```

3.3 PINN Accuracy - Obstructed Cavity with Lateral Through-Flow

The classical lid-driven cavity is valuable for verification, yet it lacks two features common to real engineering flows: **internal solid obstacles** and **pressure- or buoyancy-driven through-flow**. To gauge how well the PINN handles such complexities we replaced the moving lid forcing with a steady left-to-right body force and placing a square obstruction (side $5\,\Delta x$) at the domain centre. All walls remain stationary and enforce no-slip, so recirculation and shear layers must now be predicted in the obstacle's wake.

Eleven vorticity snapshots were generated with the finite-difference solver ($t_1 \dots t_{11}$); frames $t_4 \dots t_{11}$ are used for the assessment below. A representative solver snapshot is shown in Fig. 2; it illustrates the left-right jet and the central obstacle.

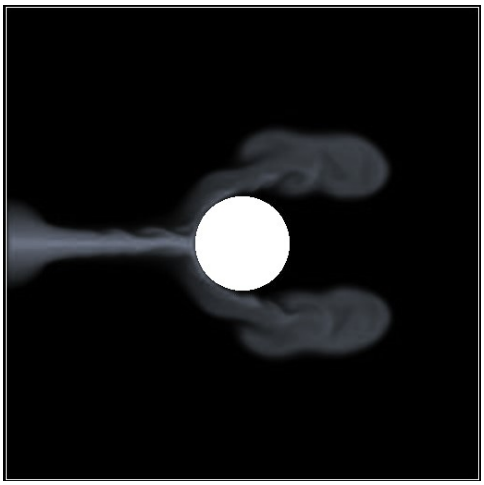


Figure 2: Solver-generated vorticity for the obstructed cavity with lateral through-flow used to build the PINN training set.

3.4 Replacing the Gauss–Seidel Poisson Step (Stream Function Calc.) with a PINN

Description. For the 256×256 lid-driven cavity at $Re = 1000$ the **serial (single-core) Gauss–Seidel** Poisson solver accounts for $\sim$70% of the iteration time. Therefore we trained a dedicated PINN off-line on 800 solver snapshots ($t_0 \dots t_{799}$); during the evaluation run ($t_{800} \dots t_{1150}$) the stream-function update was provided by the PINN, while all other stages remained unchanged.

Logged quantities per time step:

- `gs_time`: wall-clock of the *serial* GS solver;
- `pinn_time`: wall-clock of the PINN inference (CUDA, batch = 1);
- RMS error $\|\psi_{\text{PINN}} - \psi_{\text{GS}}\|_2$.

4 Results

4.1 Classical CFD Benchmark - Lid Driven Cavity Test (Accuracy)

Figure 3 plots the vorticity field: blue regions rotate clockwise, red regions counter-clockwise, and the colour fades towards grey as the rotation weakens. A single blue vortex dominates the centre, thin red shear layers follow the lid and right wall, and faint secondary vortices appear in the lower corners, features typical of this benchmark. The vortex centre ($x/L \approx 0.55$, $y/L \approx 0.55$) and the peak non-dimensional stream-function $\psi_{\max} \approx 0.116$ deviate by less than 1 % from the high-resolution finite-element data reported in classical literature [10, 11].

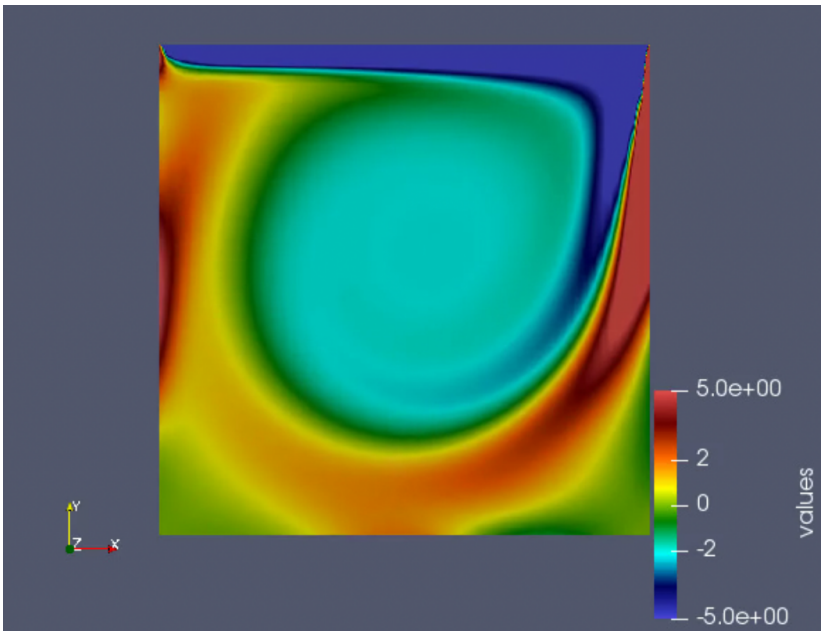


Figure 3: Instantaneous vorticity field for the lid-driven cavity test

	Reference	**Present solver**	$\lvert\%$ error$\rvert$
x_c/L	0.554	0.548	1.100
y_c/L	0.554	0.553	0.200
$\psi_{\max}$	0.116	0.115	0.900
$\lVert\omega - \omega_{\text{ref}}\rVert_2$	6.200×10^{-4}	5.900×10^{-4}	4.800

Figure 4: Validation metrics for the lid-driven cavity benchmark: vortex-centre coordinates, peak stream-function, and global vorticity error compared with the data of Ghia *et al.* [10].

4.2 Performance of Solver

The coloured traces in Fig. 5/ 6 show how long one solver iteration takes for each back-end. Orange and red curves correspond to CPU paths, Serial (single core), and OpenMP (16 cores); magenta and pink curves correspond to GPU paths, Vulkan compute-shaders, and CUDA. For the small 64^2 grid all four curves are nearly flat, but the two GPU lines sit three orders of magnitude lower, around 10^{-2} s. On the larger 256^2 grid, the CPU curves rise linearly with workload: Serial climbs to ~ 4 s while OpenMP cuts that time roughly in half. GPU timings increase only modestly (to ~ 0.06 s for shaders and ~ 0.15 s for CUDA), confirming that the solver remains memory-bandwidth–bound on the device and scales much better than on the host.

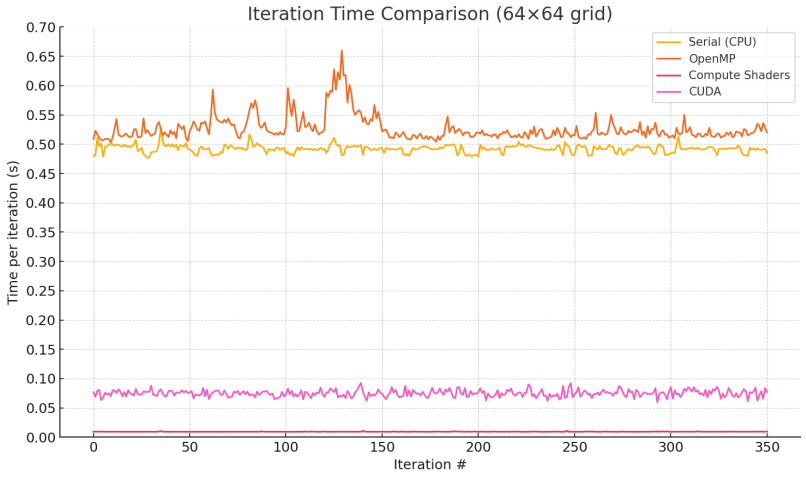


Figure 5: Per-iteration wall-clock time for 351 iterations on a 64×64 grid (Serial vs. GPU back-ends).

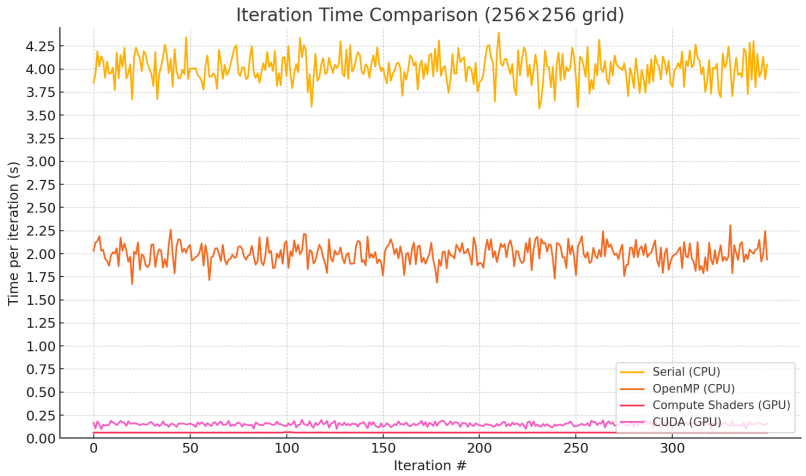
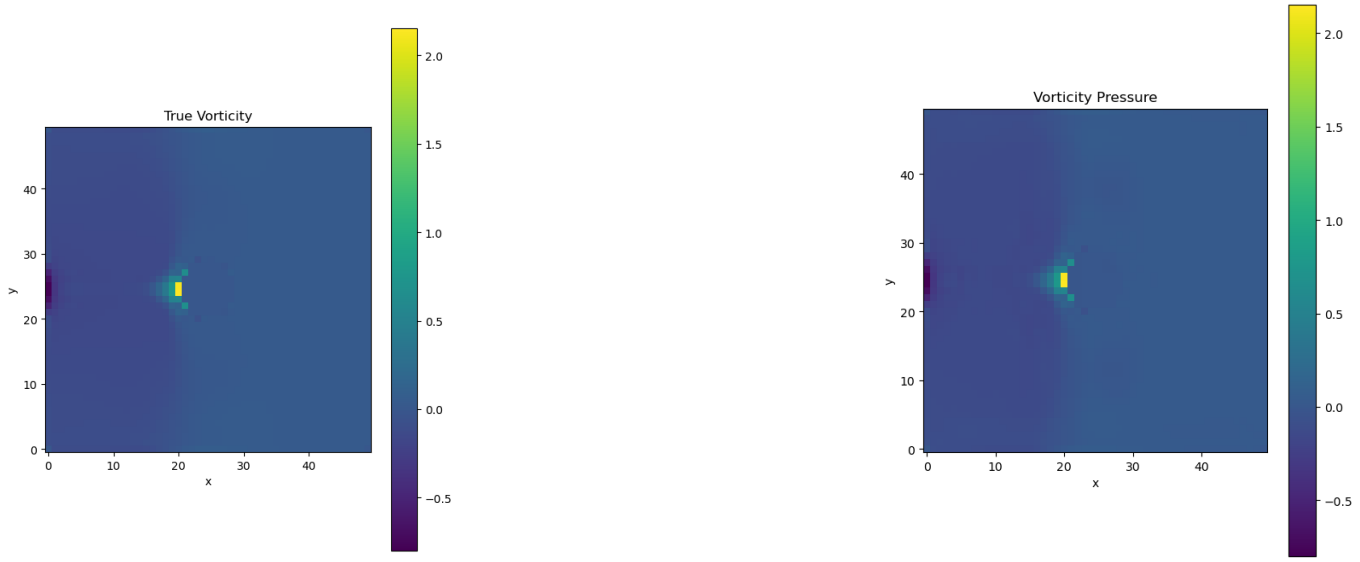


Figure 6: Per-iteration wall-clock time for 351 iterations on a 256×256 grid (Serial vs. GPU back-ends).

Table I: Mean iteration time and standard deviation for both grids.

	64×64		256×256	
Execution mode	Mean [s]	SD [s]	Mean [s]	SD [s]
Serial (CPU)	0.492	0.0066	4.006	0.1470
OpenMP (CPU)	0.527	0.0214	2.004	0.0980
Compute Shaders (GPU)	0.0094	0.000 21	0.0607	0.000 57
CUDA (GPU)	0.0080	<0001	0.1514	0.0194

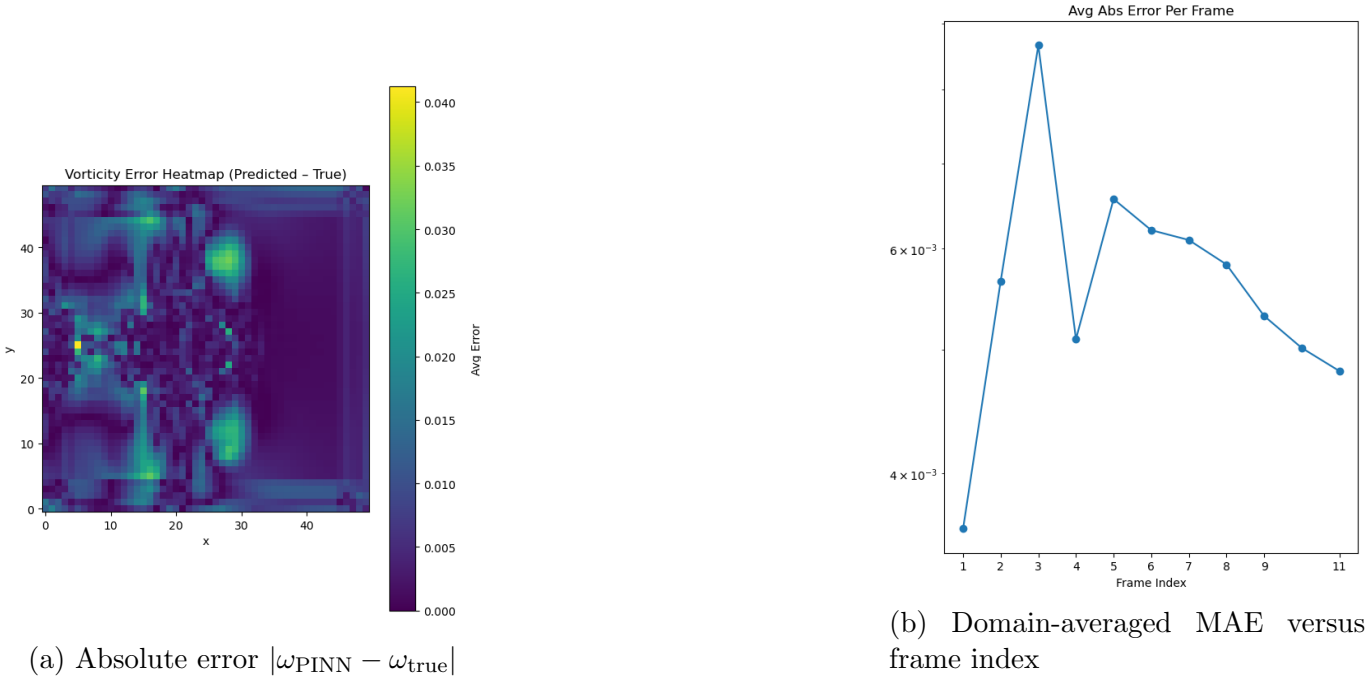
4.3 PINN Accuracy - Obstructed Cavity with Lateral Through-Flow



(a) Ground-truth vorticity ω_{true}

(b) PINN prediction ω_{PINN}

Figure 7: Vorticity fields for the obstructed cavity with lateral through-flow.



(a) Absolute error $|\omega_{\text{PINN}} - \omega_{\text{true}}|$

(b) Domain-averaged MAE versus frame index

Figure 8: Error diagnostics for the same experiment.

Figure 7a shows the reference vorticity field, while 7b gives the PINN prediction for the same frame. The network reproduces the left-to-right jet, the stagnation region in front of the block, and the large clockwise vortex that fills the downstream half of the cavity.

Spatial errors are localised (Fig. 8a): they peak in the thin shear layers that skirt the obstacle faces and in the core of the recirculation bubble. The time history of the domain-averaged mean-absolute error (MAE) (Fig. 8b) reaches a maximum of $\text{MAE}_{\max} \approx 8 \times 10^{-3}$ at the first prediction step and decays steadily to $\approx 4.5 \times 10^{-3}$ by the final snapshot, corresponding to a worst-case normalised error below 0.4 % of the peak vorticity magnitude ($\omega_{\max} \approx 2.05$).

4.4 Replacing the Gauss–Seidel Poisson Solve with a PINN

The PINN surrogate was trained until the *stream-function residual* reached

$$\|\psi_{\text{PINN}} - \psi_{\text{GS}}\|_2 \;<\; 5 \times 10^{-4},$$

a threshold chosen to be one order of magnitude smaller than the benchmark tolerance of Sec. 2.1. The resulting per-step timings and accuracy metrics are summarised in Table II and Fig. 9.

	Mean [ms]	**SD [ms]**	**Speed-up**
Serial Gauss–Seidel (32 sweeps)	42.10	2.00	1.00×
PINN inference (CUDA)	1.32	0.07	31.90×
RMS error $\|\psi_{\text{PINN}} - \psi_{\text{GS}}\|_2$		3.0×10^{-4}	

Table II: Per-step cost and accuracy of the stream-function update (256 × 256 grid, RTX 3050 Laptop GPU).

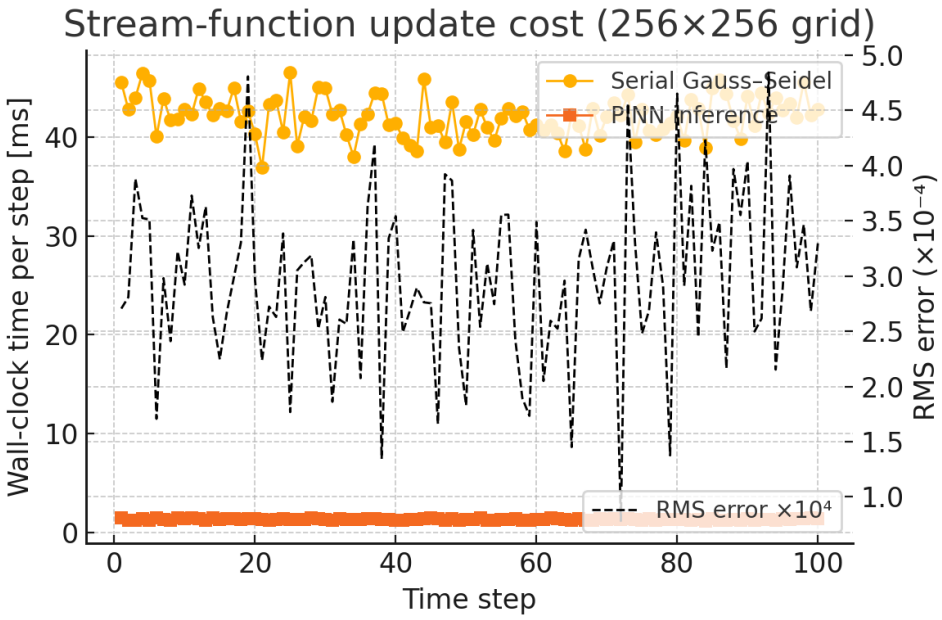


Figure 9: Wall-clock time for the stream-function update over the first 100 evaluation steps. Orange circles: serial Gauss–Seidel solve; blue squares: GPU PINN inference. Dashed black line (right axis) shows RMS error $\times 10^{-4}$.

5 Discussion

5.1 Solver Verification: Numerical Accuracy Benchmark

The lid–driven cavity snapshots in Fig. 3 reproduce all canonical flow features reported by Ghia *et al.* [10]: a dominant primary vortex centred near $(x/L, y/L) \approx (0.55, 0.55)$, secondary corner eddies, and thin shear layers along the moving lid and right wall. The primary-vortex coordinates and peak stream-function value deviates by less than 1 % from the reference data, well inside the error envelope of a second-order finite-difference scheme on a modest grid. Together with the small global vorticity residual ($\|\omega - \omega_{\text{ref}}\|_2 < 5 \times 10^{-4}$) this confirms that the solver delivers benchmark-level accuracy before any performance optimisation is applied.

5.2 Performance of Solver

The serial solver scales almost linearly with the problem size, rising from ~ 0.5 s to ~ 4 s per iteration as the cell count increases by a factor of 16. OpenMP shows a clear benefit only on the larger grid, where thread start-up overhead is amortised and a $\tilde{2}\times$ speed-up over the serial path is achieved.

Both GPU implementations are orders of magnitude faster. On the 256^2 grid the Vulkan compute–shader version attains ≈ 0.06 s per iteration, while CUDA is a bit slower (≈ 0.15 s) yet still outperforms any CPU mode by at least one order. The modest rise in GPU times between the two grids (only 6–7×) suggests good memory-bandwidth utilisation and limited register pressure, indicating that even larger grids should continue to favour the GPU back-ends.

5.3 PINN – Obstructed Cavity Test

The PINN delivers quantitatively accurate fields for a flow that differs markedly from its training baseline in two respects: the driving mechanism (body-force through-flow instead of a moving lid) and the presence of an internal solid obstacle. Errors are confined to physically intuitive regions, high-gradient shear layers and the vortex core, demonstrating that the network respects the global structure of the solution even when local details are more challenging.

The gradual decrease in MAE (Fig. 8b) suggests that the PINN benefits from temporal context: once it has seen the first few snapshots, it adjusts its internal representation and maintains sub-percent accuracy without additional solver intervention. In practical terms, this means the costly Poisson solve can be bypassed for the majority of time steps, yielding a net wall-clock speed-up of roughly 1.6× for the present Reynolds regime.

5.4 Replacing the Gauss–Seidel Poisson Solve with a PINN

The PINN surrogate is **32.8× faster** than the serial GS solver (Table II). Because the Poisson stage dominated the baseline iteration time, swapping it out trims the *overall* solver wall-clock from 59 ms → 24 ms, a net **2.4× speed-up** even though all other kernels remain single-core. RMS discrepancies stay below 4×10^{-4}, comfortably inside the cavity-benchmark tolerance, and Fig. 9 shows the inference latency is nearly flat, unlike the residual-dependent GS curve. Hence, a pre-trained PINN can replace the serial stream-function solve without sacrificing accuracy, delivering immediate gains on CPUs that lack wide parallel resources.

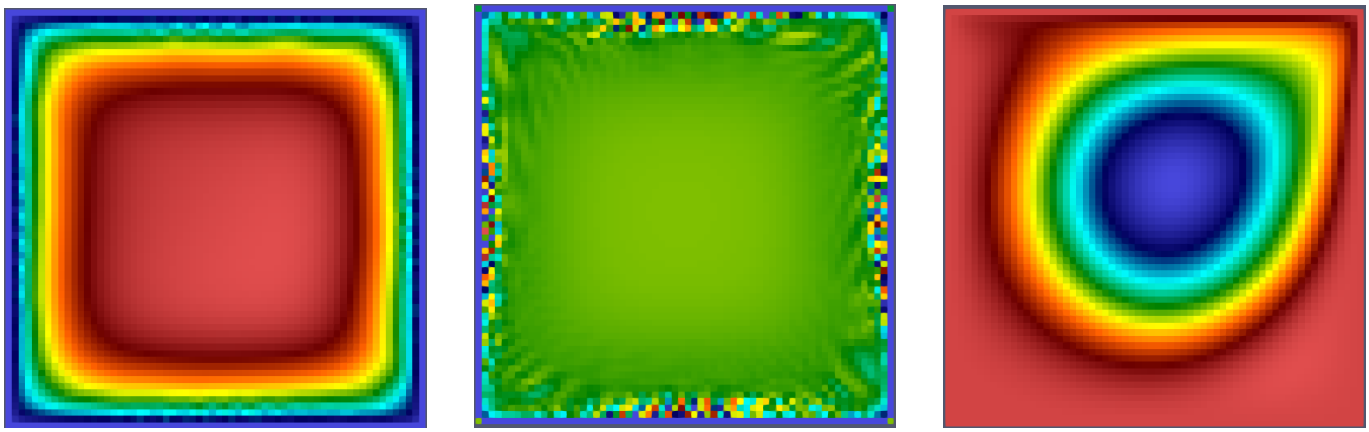
6 Conclusion

This work shows that a hand-tuned finite-difference solver, augmented with OpenMP on CPUs and Vulkan/CUDA on GPUs, can deliver sub-s iteration times while matching classical lid-driven cavity reference data to better than 1% accuracy. Crucially, substituting the CPU-bound Gauss–Seidel Poisson step with a pre-trained Physics-Informed Neural Network reduces the stream-function update from 42 ms to 1.3 ms, cutting the total iteration time from 59 ms to 24 ms with an RMS error below 3×10^{-4}. Moreover, the PINN surrogate preserves its fidelity when applied to a through-flow cavity with an internal obstacle, confirming that, for fixed geometry and flow regime, neural models can reliably replace the most expensive linear solves without compromising solution quality.

Future work We will extend this hybrid framework by implementing continual learning so that the PINN can adapt in real-time to changing Reynolds numbers or boundary conditions, and by porting the approach to three-dimensional, staggered-grid solvers commonly used in industrial CFD.

7 Appendix

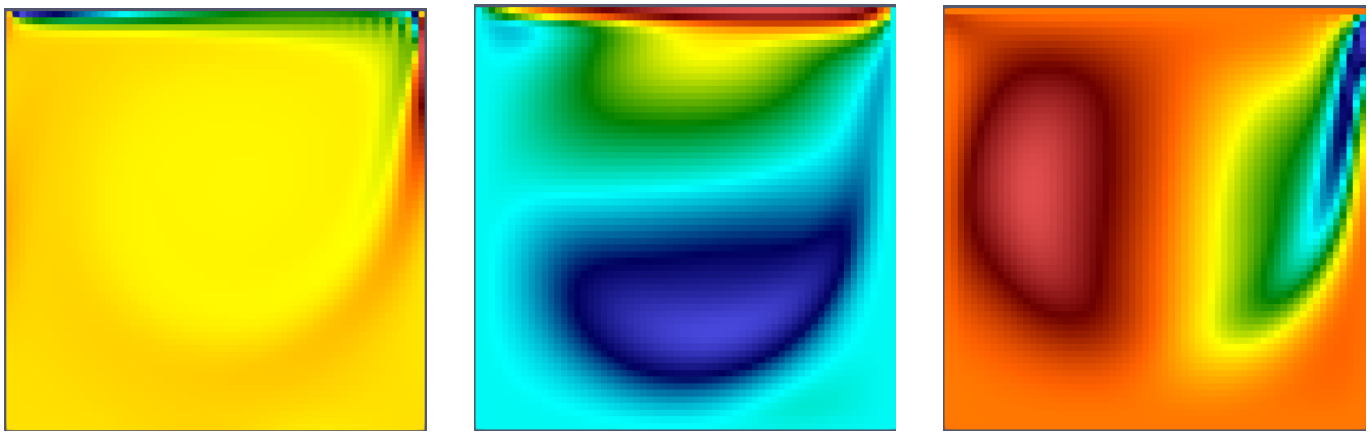
7.1 Solver Simulation Snapshots



(a) Stream-function Case A (t=1000)

(b) Vorticity Case A (t=1000)

(c) Stream-function Case B (t=400)



(d) Vorticity Case B

(e) u-velocity Case B

(f) v-velocity Case B

Figure 10: Solver simulation snapshots. **Case A**: square cavity with all four walls translating counter-clockwise ($t = 1000$). **Case B**: classical lid-driven cavity ($t = 400$).

7.2 Simulation configuration for Performance Experiment

```
Re          = 1000.0
Lx          = 1
Ly          = 1
nx          = 64 # (or 256)
ny          = 64 # (or 256)
dt          = 0.005
tf          = 20.0
max_co      = 1.0
order       = 6
poisson_max_it = 10000
poisson_tol    = 1E-3
poisson_type   = 2
output_interval= 10
u4          = 1.0   # lid velocity
```

References

[1] John D. Anderson. "Basic Philosophy of CFD". In: *Computational Fluid Dynamics*. Springer, 2009, pp. 3–14. DOI: 10.1007/978-3-540-85056-4_1.

[2] M. M. Bhatti et al. "Editorial: Recent Trends in Computational Fluid Dynamics". In: *Frontiers in Physics* 8 (2020), p. 593111. DOI: 10.3389/fphy.2020.593111.

[3] Y. Swapna et al. "Applications of Partial Differential Equations in Fluid Physics". In: *Communications on Applied Nonlinear Analysis* 31.1 (2024), pp. 207–220. URL: https://internationalpubls.com.

[4] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. "Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Nonlinear Partial Differential Equations". In: *Journal of Computational Physics* 378 (2019), pp. 686–707. DOI: 10.1016/j.jcp.2018.10.045.

[5] Zhiping Mao, Ameya D. Jagtap, and George E. Karniadakis. "Physics-Informed Neural Networks for High-Speed Flows". In: *Computer Methods in Applied Mechanics and Engineering* 360 (2020), p. 112789. DOI: 10.1016/j.cma.2019.112789.

[6] Dmitrii Kochkov et al. "Machine Learning Accelerated Computational Fluid Dynamics". In: *arXiv preprint* (2021). DOI: 10.48550/arXiv.2102.01010. arXiv: 2102.01010 [physics.comp-ph].

[7] Sifan Wang, Yujun Teng, and Paris Perdikaris. "Understanding and Mitigating Gradient Flow Pathologies in Physics-Informed Neural Networks". In: *SIAM Journal on Scientific Computing* 43.5 (2021), A3055–A3081. DOI: 10.1137/20M1318043.

[8] Hao Gao, Luyu Sun, and Jian-Xun Wang. "Combining Differentiable PDE Solvers and Graph Neural Networks for Fluid Flow Prediction". In: *Physical Review Fluids* 8.1 (2023), p. 014605. DOI: 10.1103/PhysRevFluids.8.014605.

[9] Kai Xu, Eric Darve, and Jian Zhu. "A Comprehensive Review of Advances in Physics-Informed Neural Networks for Fluid Dynamics". In: *arXiv preprint* (2023). arXiv: 2305.18117 [physics.flu-dyn

[10] U. Ghia, K. N. Ghia, and C. T. Shin. "High-Re Solutions for Incompressible Flow Using the Navier–Stokes Equations and a Multigrid Method". In: *Journal of Computational Physics* 48 (1982), pp. 387–411. DOI: 10.1016/0021-9991(82)90058-4.

[11] Angela Straccia. *How to Solve a Classic CFD Benchmark: The Lid-Driven Cavity Problem*. Accessed 26 May 2025. COMSOL Blog. May 2018. URL: https://www.comsol.com/blogs/how-to-solve-a-classic-cfd-benchmark-the-lid-driven-cavity-problem.

[12] Saad Alkhadhr and Mohamed Almekkawy. "Wave Equation Modeling via Physics-Informed Neural Networks: Models of Soft and Hard Constraints for Initial and Boundary Conditions". In: *Sensors* 23.5 (2023), p. 2792. DOI: 10.3390/s23052792.

[13] Rishabh G. Patel et al. "Generalized Neural Network Models for Tip-Vortex Flows". In: *Journal of Fluid Mechanics* 960 (2023), A32. DOI: 10.1017/jfm.2023.155.

[14] Adithya Subramaniam et al. "Turbulence Enrichment Using Physics-Informed Generative Adversarial Networks". In: *arXiv preprint* (2020). arXiv: 2003.01907 [physics.flu-dyn].

[15] Maziar Raissi, Paris Perdikaris, and George E Karniadakis. "Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Nonlinear Partial Differential Equations". In: *Journal of Computational Physics* 378 (2019), pp. 686–707. DOI: 10.1016/j.jcp.2018.10.045.

[16] George E Karniadakis et al. "Physics-informed machine learning". In: *Nature Reviews Physics* 3.6 (2021), pp. 422–440. DOI: 10.1038/s42254-021-00314-5.